
name: code-review

description: Review the changes since a fixed point (commit, branch, tag, or merge-base) along two axes — Standards (does the code follow this repo's documented coding standards?) and Spec (does the code match what the originating issue/spec asked for?). Runs both reviews in parallel sub-agents and reports them side by side.

Use when the user wants to review a branch, a PR, work-in-progress changes, or asks to "review since X".

Two-axis review of the diff between HEAD and a fixed point the user supplies:

- Standards — does the code conform to this repo's documented coding standards?
- Spec — does the code faithfully implement the originating issue / spec?

Both axes run as parallel sub-agents so they don't pollute each other's context, then this skill aggregates their findings. The issue tracker should have been provided to you — run `/setup-matt-pocock-skills` if `docs/agents/issue-tracker.md` is missing.

Process

1. **Pin the fixed point**.

Whatever the user said is the fixed point — a commit SHA, branch name, tag, ``main``, ``HEAD~5``, etc. If they didn't specify one, ask for it.

Capture the diff command once: ``git diff ...HEAD`` (three-dot, so the comparison is against the merge-base).

Also note the list of commits via ``git log ..HEAD --oneline``.

Before going further, confirm the fixed point resolves (``git rev-parse``) and the diff is non-empty. A bad ref or empty diff should fail here — not inside two parallel sub-agents.

2. **Identify the spec source**.

Look for the originating spec, in this order:

- Issue references in the commit messages (``#123``, ``Closes #45``, etc.)
- A path the user passed as an argument.
- A spec file under ``docs/``, ``specs/``, or ``scratch/`` matching the branch name or feature.

3. **Spawn Parallel Review Agents**:

- Agent 1 (**Standards**): Analyzes the diff against ``CODING_STANDARDS.md`` or a baseline of Fowler code smells (Mysterious Name, Duplicated Code, Feature Envy, etc.).
- Agent 2 (**Spec**): Analyzes whether the code actually satisfies the requirements from the spec or issue tracker without smuggling in scope creep.

4. **Aggregate Results**:

Output the findings side-by-side under `## Standards` and `## Spec` headers without blending the verdicts together.